

Laboratorium 9. Modelowanie wymagań niefunkcjonalnych.

Metryki jakości oprogramowania.

Sviatoslav Protsyk 143540

Zadanie 5. Niezawodność oprogramowania (Software Reliability)

1. Definicja pojęcia

Niezawodność oprogramowania (Software Reliability) to prawdopodobieństwo bezawaryjnego działania systemu komputerowego w określonym środowisku oraz w określonym przedziale czasowym. W przeciwieństwie do niezawodności sprzętu, nie wynika ona ze zużycia materiału, lecz z ukrytych błędów projektowych i implementacyjnych, które aktywują się w specyficznych warunkach uruchomieniowych.

2. Metody specyfikacji wymagań

- **POFOD (Probability of Failure on Demand):** Prawdopodobieństwo, że system ulegnie awarii w momencie zgłoszenia żądania obsługi (np. $POFOD = 0.001$ oznacza 1 awarię na 1000 żądań).
- **ROCOF (Rate of Occurrence of Failures):** Częstotliwość pojawiania się awarii w jednostce czasu (np. $ROCOF = 0.02$ oznacza średnio 2 awarie na 100 godzin pracy).
- **MTTF (Mean Time To Failure):** Średni czas bezawaryjnej pracy systemu do momentu wystąpienia pierwszej awarii.
- **MTBF (Mean Time Between Failures):** Średni czas pomiędzy kolejnymi awariami (suma MTTF oraz średniego czasu naprawy MTTR).

3. Test Case – Weryfikacja Niezawodności Usługi WWW (Soak Testing / Test Stabilności)

ID / Nr Testu	TC_REL_001
Nazwa testu	Test funkcjonalno-niezawodnościowy pod długotrwałym obciążeniem (Soak Test)
Cel testu	Weryfikacja parametru $ROCOF < 0.01$ na dobę oraz stabilności zużycia pamięci RAM/CPU przez usługę WWW przy ciągłym obciążeniu bazowym.
Dane testowe	Skrypt automatyczny Apache JMeter generujący stały ruch na poziomie 50 jednoczesnych użytkowników (Virtual Users) wykonujących operacje przeglądania i zakupu.
Warunki wstępne	Środowisko stagingowe tożsame z produkcyjnym. Monitorowanie zasobów (Prometheus/Grafana) włączone. Brak błędów w logach przed uruchomieniem testu.
Opis kroków i przebieg testu	

1. Uruchom skrypt generujący stałe obciążenie (50 VU) na okres 48 godzin bez przerwy. 2. Monitoruj liczbę odpowiedzi HTTP 5xx (błędy serwera) oraz HTTP 4xx wywołanych awariami aplikacji. 3. Weryfikuj poziom użycia pamięci RAM na serwerze aplikacyjnym co 2 godziny w celu wykrycia wycieków pamięci (Memory Leaks).	
--Oczekiwany wynik--	System działa stabilnie przez 48 godzin. Łączna liczba awarii/błędów aplikacji wyniosła 0 (ROCOF = 0). Wykres zużycia pamięci RAM nie wykazuje trendu rosnącego (brak wycieków pamięci). Wynik końcowy: Pozytywny.

Zadanie 6. Obsługiwalność systemu (Maintainability)

1. Definicja pojęcia

Obsługiwalność (Maintainability) określa łatwość, z jaką system oprogramowania może być modyfikowany przez programistów i administratorów. Modyfikacje te obejmują usuwanie defektów, dostosowywanie do nowych środowisk uruchomieniowych, refaktoryzację oraz rozbudowę o nowe funkcjonalności.

2. Metody specyfikacji wymagań

- **MTTR (Mean Time To Repair):** Średni czas potrzebny na zdiagnozowanie, naprawienie i przetestowanie poprawki po wystąpieniu awarii (np. $MTTR < 2$ godziny).
- **Pokrycie kodu testami (Code Coverage):** Wymagany procent linii kodu pokryty testami automatycznymi (np. $Coverage > 80\%$).
- **Złożoność cykliczna (Cyclomatic Complexity):** Ograniczenie stopnia skomplikowania kodu źródłowego (np. maksymalna złożoność dla pojedynczej metody < 10).

3. Test Case – Weryfikacja Obsługiwalności (Automatyzacja Wdrożeń CI/CD i Naprawy)

ID / Nr Testu	TC_MAINT_001
Nazwa testu	Weryfikacja czasu wdrożenia poprawki awaryjnej (Hotfix Deployment Time)
Cel testu	Weryfikacja czy system pozwala na automatyczne zbudowanie, przetestowanie i wdrożenie poprawki (Hotfix) w czasie krótszym niż 15 minut przy użyciu potoku CI/CD.
Dane testowe	Zmiana w kodzie źródłowym symulująca naprawę błędu (np. zmiana tekstu komunikatu w module logowania) w repozytorium Git.

Warunki wstępne	Skonfigurowane repozytorium Git oraz działające narzędzie CI/CD (np. GitHub Actions / GitLab CI). Dostęp do serwera testowego.
Opis kroków i przebieg testu	
1. Wykonaj 'Commit' i 'Push' poprawki do gałęzi produkcyjnej (main/master). Włącz stoper. 2. Monitoruj automatyczne uruchomienie potoku (Pipeline): budowanie aplikacji, uruchomienie testów jednostkowych i integracyjnych. 3. Zatrzymaj stoper w momencie automatycznego wdrożenia nowej wersji oprogramowania na środowisko testowe i zwrócenia statusu 'Success'.	
--Oczekiwany wynik--	Potok CI/CD wykonuje się bezbłędnie. Całkowity czas od wrzucenia kodu do zakończenia wdrożenia wynosi mniej niż 15 minut (np. 8 minut 45 sekund). Kod pomyślnie przechodzi statyczną analizę jakości. Wynik końcowy: Pozytywny.

Zadanie 7. Dostępność systemu (Availability)

1. Definicja pojęcia

Dostępność (Availability) to stosunek czasu, w którym system jest w pełni zdalny do użytku i zdolny do realizowania swoich zadań, do całkowitego czasu, w którym system powinien działać. Jest ona ściśle powiązana zarówno z niezawodnością (jak rzadko się psuje), jak i obsługiwalnością (jak szybko można go naprawić). Wyraża się ją wzorem matematycznym:

$$A = MTBF / (MTBF + MTTR)$$

2. Metody specyfikacji wymagań

Najczęstszą metodą specyfikacji jest określenie poziomu procentowego w umowach SLA (Service Level Agreement):

- **99.9% ("trzy dziewiątki"):** Dopuszczalny przestój w roku to maksymalnie 8 godzin 45 minut.
- **99.99% ("cztery dziewiątki"):** Dopuszczalny przestój w roku wynosi maksymalnie 52 minuty i 35 sekund.

3. Test Case – Weryfikacja Dostępności (Failover / High Availability Test)

ID / Nr Testu	TC_AV_001
Nazwa testu	Test odporności na awarię węzła serwera (High Availability / Failover Test)
Cel testu	Weryfikacja zachowania ciągłości działania usługi WWW (dostępności) w przypadku nagłej awarii jednego z dwóch głównych serwerów aplikacji pracujących za Load Balancerem.
Dane testowe	Ruch ciągły HTTP generowany przez skrypt testowy wysyłający 10 żądań na

	sekundę.
Warunki wstępne	System działa w architekturze rozproszonej (klaster z dwoma instancjami oprogramowania Node_A i Node_B) podpiętymi pod mechanizm równoważenia obciążenia (Load Balancer).
Opis kroków i przebieg testu	
1. Uruchom ciągle wysyłanie żądań HTTP do adresu głównego usługi WWW. 2. W trakcie trwania ruchu, zasymuluj nagłą awarię poprzez twarde wyłączenie procesu aplikacji na serwerze Node_A (np. komenda kill -9). 3. Przeanalizuj logi Load Balancera oraz skryptu wysyłającego żądania pod kątem wystąpienia błędów połączenia (HTTP 502/503) oraz zmierz czas przełączenia ruchu na działający serwer Node_B.	
--Oczekiwany wynik--	Load Balancer natychmiast (w czasie < 1 sekundy) wykrywa awarię Node_A i przekierowuje cały ruch do Node_B. Użytkownik końcowy nie zauważa przerwy w działaniu strony. Liczba utraconych żądań wynosi 0, a usługa zachowuje 100% dostępności w trakcie incydentu. Wynik: Pozytywny.

Zadanie 8. Różnica między awarią (failure) a błędem (fault)

W inżynierii jakości oprogramowania (zgodnie ze standardem słownictwa ISTQB/IEEE) pojęcia te reprezentują zupełnie inne etapy i przejawy problemu w systemie:

Błąd / Defekt (Fault / Defect)	Awaria (Failure)
Jest to statyczna wada w kodzie źródłowym lub dokumentacji oprogramowania. Powstaje na skutek pomyłki ludzkiej (błędu programisty, architekta lub analityka, zwanego jako <i>error</i>).	Jest to dynamiczne zachowanie oprogramowania, czyli widoczne dla użytkownika odchylenie działania systemu od jego oficjalnej specyfikacji lub oczekiwań.
Przykład: Programista wpisuje w kodzie błędny znak w warunku pętli: if (i > 10) zamiast prawidłowego if (i >= 10). Błąd ten tkwi w kodzie na dysku i sam z siebie niczego nie psuje, dopóki ta linijka nie zostanie uruchomiona.	Przykład: Użytkownik klika przycisk na stronie, uruchamia się wadliwy kod, pętla wykonuje się o jeden raz za mało, w wyniku czego system zawiesza się lub wyświetla komunikat "Błąd systemu". Użytkownik doświadcza awarii.

Podsumowując zależność przyczynowo-skutkową:

Człowiek popełnia pomyłkę (**Error**) → Tworzy to wadę/defekt w kodzie (**Fault/Defect**) → Podczas uruchomienia programu ta wada zostaje wykonana → System zachowuje się niepoprawnie i następuje awaria (**Failure**).